

Maven Basics

What is Maven, and what problem does it solve in a Java project?

A: **Maven** is a project management tool for Java projects. It solves the problems of **dependency management** (automatically downloading the libraries your project needs) and **standardizing the build process**. This ensures that any developer can easily build any project using the same set of simple rules.

What is the pom.xml (POM file) in Maven, and why is it important?

A: The **pom.xml (Project Object Model file)** is the main configuration file for every Maven project. It is important because it defines all the essential details about your project, such as its name, version, what libraries it needs (dependencies), and how to build it using various plugins.

What is a Maven repository, and what are the different types (local, central, remote)?

A: A Maven **repository** is a dedicated place where all project libraries and artifacts (JAR files) are stored. There are three types:

1. **Local:** The cache on your personal computer.
2. **Central:** The main public repository on the internet managed by Apache.
3. **Remote:** Any private or organization-specific server used to store company libraries.

What is the settings.xml file in Maven, and what is it used for?

A: The **settings.xml** file is used to store user-specific or machine-specific configurations for Maven. It stores things that should **not** be checked into the project's main pom.xml, such as your personal server passwords or specific network proxies needed to download libraries.

Dependencies & Resolution

What is the difference between compile-time and runtime dependencies in Maven?

A: **Compile-time dependencies** are libraries needed both to write/compile your code and to run the final application (e.g., Spring Core). **Runtime dependencies** are only needed when the application is actually running, but the compiler does not need them (e.g., a database JDBC driver).

How can you exclude a specific transitive dependency in Maven?

A: You exclude a transitive dependency (a library pulled in by another required library) by using the **<exclusion>** tag within the main dependency definition in the pom.xml. This is done to prevent version conflicts or to remove unwanted libraries from your final project package.

What are dependency scopes in Maven, and when would you use them?

A: Dependency scopes define **when** a dependency is needed during the build process (e.g., compile, test, runtime). You use the **test** scope for libraries like JUnit that are only needed when running tests, keeping them out of the final application package. You use the **provided** scope for libraries (like a Servlet API) that the environment supplies.

How do you use the mvn dependency:tree command, and how does it help you debug dependency issues?

A: You use **mvn dependency:tree** from the command line to print a visual map showing all your project's dependencies and how they are related. It helps you debug issues by quickly revealing where an unwanted or conflicting version of a transitive dependency is coming from.

Maven Lifecycle & Build

What is the Maven build lifecycle, and what are its main phases?

A: The Maven build lifecycle is a defined sequence of steps for building and deploying a project. The main lifecycle (default) includes phases like **validate** (check project), **compile** (compile code), **test** (run tests), **package** (create JAR/WAR), and **install** (copy to local repository).

What is the difference between mvn clean and mvn install?

A: **mvn clean** deletes the entire target directory, removing all previously compiled code and build artifacts. **mvn install** compiles the code, runs tests, creates the final package, and then copies that finished artifact into your local repository.

How do you run a Maven build from the command line for a typical project?

A: You run a build using the **mvn** command followed by the desired phase, such as **mvn clean install**. When you run install, Maven automatically executes all phases before it, including validate, compile, and package, in the correct order.